SUBMISSION

The submission consists of **2** files uploaded into Canvas.

- A **document** in .pdf format with the solutions and justifications to ALL the exercises. If you want to create documents by hand and scan/photograph them to add them to the pdf document, please ensure the quality is good and readable.
    - You must include a **link** to your shared **git repository** at the beginning of the document. Failure to provide the **git link** will incur in a mark of 0 in all code-based questions.
    - The code should be self-contained; i.e., done in a way such as if anyone else wants to "download" your code and run it, they can. Assume everyone has the 4 basic libraries already installed (NumPy, Matplotlib, Pandas and NetworkX) – no need to reinstall those. The git repo must have ALL the code for ALL the questions.
    - All of your code **must** be in Python.
    - If a question includes a specific function definition, you **must** use that signature. If the question specifies a format for your output, you code **must** output in that format.
    - You also **must** include all the code in the report as text.
- A **video**, preferably in .mp4 format, no longer than 10 mins, with the explanation of your solutions for questions 1 and 2. You must have your camera on and your face must be clearly visible in the video.

In this assessment, students are **expressly prohibited** from (i) using GenAI tools to create content and (ii) using GenAI tools, a proofreader, or a proofreading service to proofread their work

WEIGHTING

This submission is worth **60%** of your final mark.

As the final submission, in this assignment you will need to demonstrate your knowledge in all the content related to the module: Units 1, 2, 3, 4, 5 and 6.

The code **must be your own**; you might use functions of libraries such as "ones" or "randint" in NumPy, but the main code must be your own work.

TASKS

1. (25 marks) Code a maze solver using the maze provided in the brief as "maze(1).png". The resolution of the maze is 21 squares x 21 squares, including the outer wall. Please read the whole exercise before starting.

The entrance is at the top. Each movement must be 1 square in any orthogonal direction (up, down, left, right): diagonal movements are not allowed. The instructions are as follows:

Coding

a) Save the maze into an appropriate variable/data structure and display it in your terminal.

Hint: Matplotlib has some interesting functions to deal with images; you are allowed to use them if needed.

b) Ask the user what approach they want to use (Backtracking or Las Vegas).

b.i) When applying a Las Vegas approach, the end conditions are either you found the exit (successful) or you have completed 200 steps without finding the exit (unsuccessful).

b.ii) For the Backtracking approach, you MUST explain in the report what is your rationale for choosing a particular direction and how you do the backtracking.

c) Give a visualization of all the visited "squares" (positions) when the program ends, regardless of if it is successful or not.

Testing

Please add in the report between 3 and 5 instances of testing your programme. For each testing instance, you need to create a "bug report" with the following information:

1. What went wrong or what were you trying to fix.
2. Screenshot of the error or issue.
3. How did you fix it and why it was important to fix.

After having a complete program, please compare Las Vegas and Backtracking approaches and answer the following questions in your report:

a) What are their differences?
b) Which data structure/s would you use to implement this problem? Give your reasoning.
c) What is the success rate of each of the algorithms after 10000 runs?

Record yourself going over your code explaining the rationale, and highlighting the places of the code where you fixed it.

Hint: To know the success rate, record which runs give a successful result. Then, divide the successful number of runs over the total amount of runs.

2. (25 marks) Code and test one of the Minimum Spanning Tree (MST) algorithms, either Prim's or Kruskal's, for a connected NetworkX graph with 6 nodes and 11 edges. You must use the following signature:

```
def MyMinimumSpannigTree(Graph) -> Graph
```

This tells you that your code should contain a function called "MyMinimumSpanningTree" which receives 1 argument in the form of a NetworkX graph, and it returns 1 NetworkX graph.

Coding

The steps that you must do are below:

a) Create and depict the original connected graph using matplotlib.
b) Call the MyMinimumSpanningTree function.
    a. State which MST algorithm you are following.
    b. Show the creation of the MST step by step using the console.
c) Depict the final MST using matplotlib.

Testing

a) Do the same for **2 new graphs** that you create. The graphs must be connected and have at least 5 nodes and 8 edges, and be different to the first one and between themselves.

To summarize: in your report you should have images of 3 different graphs with their correspondent 3 different MSTs.

Record yourselves explaining the MST algorithm that you selected, citing your sources. Then, explain the rationale of your code and include a **clip of your code running and building the MST step by step for the first graph**.

3. (20 marks) Recursion.

You must create a function that follows the following logic:

The function receives a list of strings. Each string can be either “blue”, “red” or “green”. Your goal is to move through the list incrementing the index by following the instructions below:

- If the string at index *i* is “blue”, the next element is the element **following** the **next** appearance of the string “blue”.
- If the string at index *i* is “red”, the next element is the element in the index **2*(i) + 1**.
- If the string at index *i* is “green”, the next element is the **following** element in the list, i.e. at the index **i+1**.

The output should be a list with all the indexes of the elements you visited, in order.

Example: For the list [“blue”, “green”, “red”, “blue”, “green”, “red”], the sequence of steps would be:

1. The first step is the first element, “blue”, in index 0 -> search for the next “blue” and take the following element. The next blue is in index 3, followed by “green” in index 4.
2. The second element is “green” in index 4 -> go to the next element, which is “red” in index 5.
3. The third element is “red” in index 5 -> go to the element where its index is 2 times the current index + 1. The program should go to the element in index 11, but it doesn’t exists, therefore the program exits.

The output is [0, 4, 5].

a) Code the function using **recursion**. Answers without using recursion will score a 0 in the question. Please use the following signature:

```
def follow_instructions(instructions: list[str]) -> list[int]
```

   i. Test it with 2 generated lists.

b) Create the flowchart of your code and put it in the report.

4. (20 marks) No coding question. Please solve the following string alignment problem using Needleman-Wunsch. You need to use the optimal alignment algorithm with the strings ATTACACG and CACAGAGT.

Please provide the alignment matrix, the traceback and the string alignment using the scoring present in Unit 6.1.

What would happen if the penalty for a substitution is bigger the penalty for a gap? Would that change the optimal string alignment? If so, what would it be? Please give only the new string alignment.

5. (10 marks) No coding question.

Please read the two papers presented in Unit 6.2. After reading them, please answer the following questions in your report:

a) What are the possible behaviours modeled in each paper? Explain each distinct behaviour in a couple of lines.
b) Both articles compare their models with other, pre-existing models. Choose one pre-existing that they have in common and compare the results for both models against the common pre-existing model.

5.

MARKING SCHEME

The submission will be marked to a maximum of 100 marks. A breakdown of the marking scheme can be found below. There is a rubric available as well.

Exercise	Marks
Maze Solver	25
Minimum Spanning Tree	25
Most Frequent Word	20
Needleman-Wunsch	20
Bio-inspired Algorithms	10